

# **Dominando Decoradores em Python**

Um guia visual sobre funcionamento interno,  
padrões de projeto e casos de uso avançados.



# Sumário

1. Fundamentos: O Padrão **Decorator** (GoF)

2. A Mecânica Visual: Interface e Transparência

3. Sintaxe: **Açúcar Sintático** e **Anatomia**

4. Fluxo de Dados: **Parâmetros** e **Empilhamento**

5. Escopo Expandido: **Métodos**, **Classes** e **Singleton**

6. Ciclo de Vida: **Import Time** vs. **Runtime**

7. Preservação de Metadados (**Functools**)

8. Resumo: **Guia de Bolso**



# O que são Decoradores?

Um padrão de projeto clássico transformado em ferramenta nativa.

- **Origem:** Baseado no livro Design Patterns do Gang of Four (GoF).
- **Propósito:** Adicionar comportamento extra a objetos específicos sem a necessidade de criar subclasses (herança).
- **Em Python:** São **funções** que recebem outra função (ou chamável) como **parâmetro** e retornam uma terceira função. Atuam de forma semelhante a macros.

## A Metáfora do Embrulho

### Comportamento Adicionado

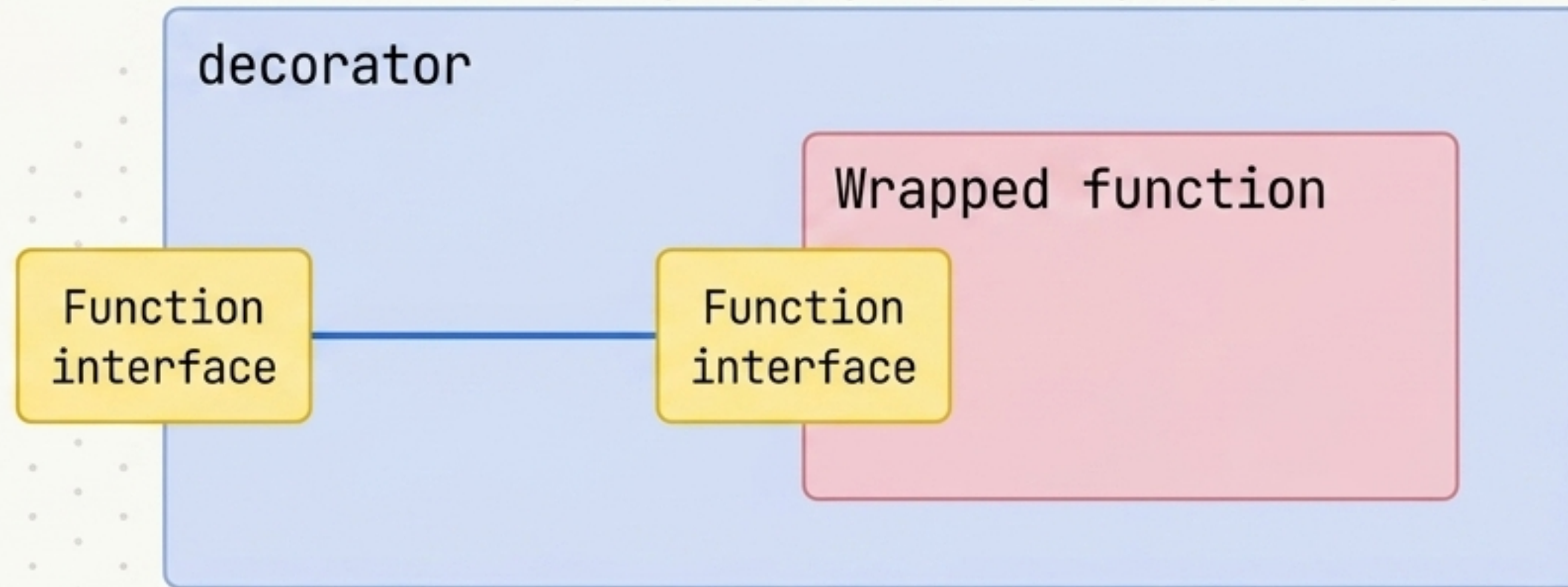
Passada como parâmetro

Função Original



# A Mecânica Visual

Transparência total para o usuário final.



**Aviso Crítico:** O decorador apresenta EXATAMENTE a mesma interface (parâmetros e retorno) que a função original. Para o cliente que chama a função, a decoração é invisível.



# Anatomia de um Decorador Simples

Dissecando a função `logger`.

```
1 def logger(func):  
2     def inner():  
3         print('calling', func.__name__)  
4         func()  
5         print('called', func.__name__)  
6  
7     return inner
```

**1. Função externa:** recebe o objeto alvo (`func`).

**2. Função interna (wrapper):** define o novo comportamento e encapsula a original.

**3. Execução:** a função original é invocada aqui.

**4. Retorno:** devolve a referência da função interna (SEM executá-la).



# Sintaxe: Explícita vs. Pythonica

O "Açúcar Sintático" por trás do símbolo "@`".

## Abordagem Explícita

```
def target():  
    print('In target')  
  
t1 = logger(target)  
t1()
```

Passamos a função por referência e reatribuímos manualmente.

← É EQUIVALENTE A →

## Açúcar Sintático

```
@logger  
def target():  
    print('In target')  
  
target()
```

O símbolo @ abstrai a atribuição (target = logger(target)), mantendo o código limpo e idiomático.



# Lidando com Parâmetros

A função **wrapper** deve espelhar a assinatura original.

```
def logger(func):  
    def inner(x, y): # <-- Espelho dos parâmetros  
        print('calling', func.__name__, 'with', x, y)  
        func(x, y) # <-- Repasse dos parâmetros  
    return inner  
  
@logger  
def my_func(x, y):  
    print(x, y)
```

A dashed blue line with an arrowhead points from the parameters 'x, y' in the 'inner' function definition to the parameters 'x, y' in the 'my\_func' definition, illustrating how the wrapper function passes the arguments to the original function.

**Regra:** Se a função decorada recebe argumentos, a função interna criada pelo decorador **OBRIGATORIAMENTE** deve recebê-los e repassá-los.



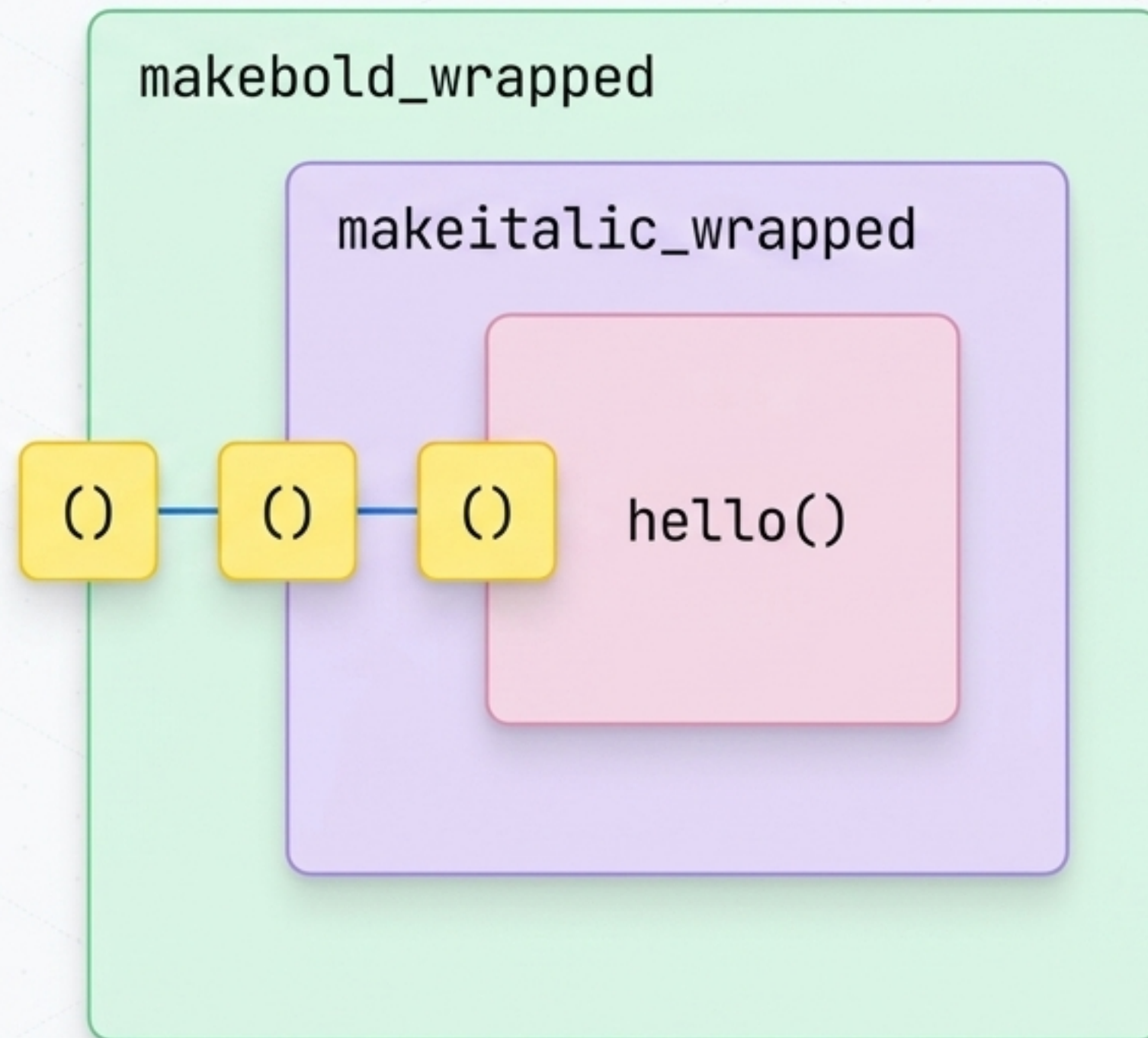
# Empilhamento (Stacked Decorators)

Múltiplas camadas de comportamento.

```
@make_bold
@make_italic
def hello():
    return 'hello world'
```

↑ Ordem de Avaliação

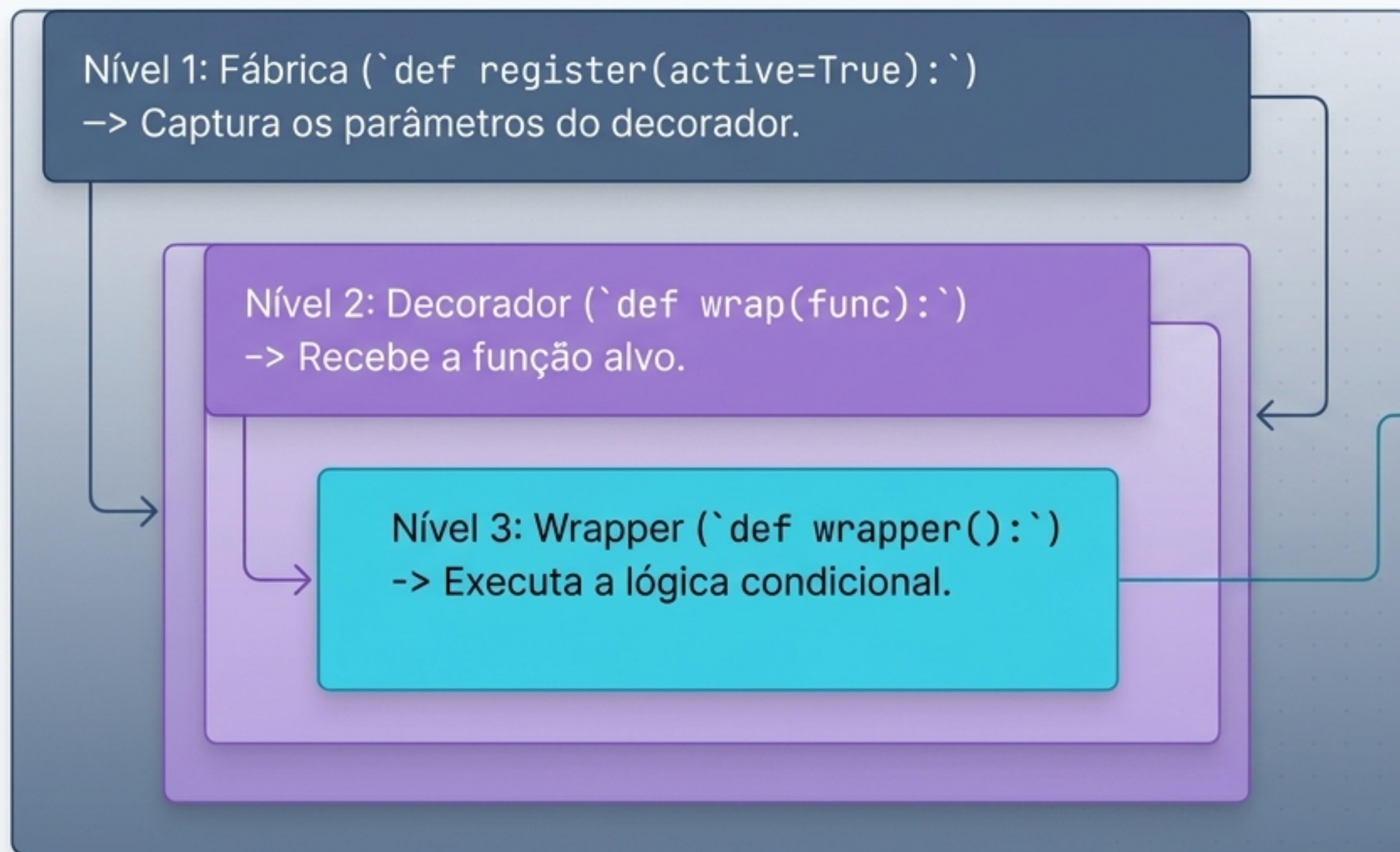
1. `hello()` é passada para `make\_italic()`.
2. O resultado (`makeitalic\_wrapped`) é passado para `make\_bold()`.
3. A chamada final executa de fora para dentro, gerando: `**<i>hello world</i></b>`.**





# Decoradores Parametrizados

Adicionando um nível extra de indireção.



**⚠ Atenção:** Mesmo ao usar valores padrão, os parênteses são obrigatórios na chamada: `@register()`.

```
@register(active=False)
def func2():
    print('func2')
```



# Expandindo os Alvos: Métodos e Classes

## Decorando Métodos

```
def trace(method):  
    def wrapper(self, x, y):  
        print('Move...')  
        method(self, x, y)  
  
    return wrapper
```

**Conceito:** É obrigatório incluir o parâmetro `self` para repassar a referência do objeto.

## Decorando Classes

```
@singleton  
class Service:  
    pass
```

**Conceito:** Substitui o construtor da classe por uma função geradora.

**Exemplo:** Padrão Singleton. Garante que apenas UMA instância exista. `s1` e `s2` apontarão para o mesmo endereço de memória hexadecimal.



# Ciclo de Vida: Import Time vs. Runtime

Quando o código do decorador é executado?

## Phase 1: Import Time (Definição)

- **Ação:** O `interpretador` avalia a `sintaxe` `@``. A função externa do `decorador` é executada IMEDIATAMENTE. A função original é `'embrulhada'`.
- **Evidência:** Um `print` solto no decorador aparecerá no console antes mesmo do programa iniciar seu fluxo principal.

## Phase 2: Runtime (Execução)

- **Ação:** A `função interna` (`wrapper`) e a `função decorada` SÓ são executadas quando explicitamente invocadas no `código` (ex: ``print_it()``).



# O Problema dos Metadados

Preservando a identidade da função original com ``functools``.

## O Problema

- **Causa:** Ao embrulhar a função, a identidade original é sobrescrita pela função interna.
- **Sintoma:** O nome e a docstring são perdidos. O debug mostra `get_text.__name__` como `'inner'`.

```
def logger(func):  
    def inner(): ...  
    return inner  
  
@logger  
def get_text(): """Doc"""  
  
print(get_text.__name__) # Output: inner
```



## A Solução

```
1 from functools import wraps  
2  
3 def logger(func):  
4     @wraps(func) # <-- Restaura metadados originais  
5     def inner(): ...  
6     return inner  
7  
8 @logger  
9 def get_text(): """Doc"""  
10  
11 print(get_text.__name__) # Output: get_text
```

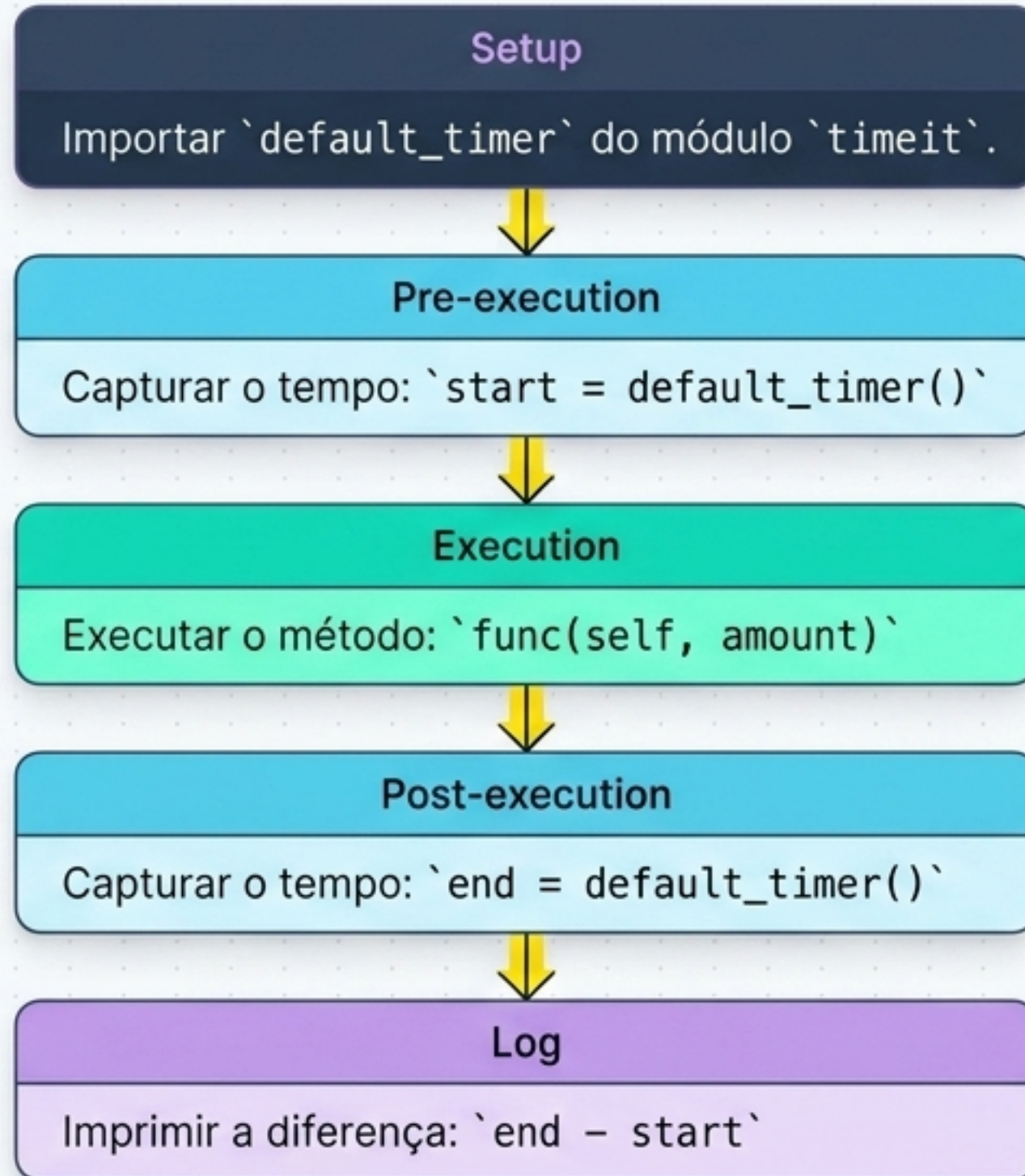


**Resultado:** A identidade correta (`get_text`) é preservada. Importado da biblioteca padrão ``functools``.



# Aplicação Prática: Criando um Decorador de Timer

Medindo a performance de métodos de uma classe.



## Exemplo de Uso

```
@timer
def deposit(self, amount):
    self._balance += amount
```

**Uso:** Gera automaticamente logs de performance em segundos, sem poluir a lógica de negócios da classe.



# Resumo: Guia de Bolso

## 1. O Conceito

Um Padrão de Projeto (GoF) implementado nativamente. **Funções** que recebem **funções** e retornam **funções** para injetar comportamento sem usar herança.

## 2. A Sintaxe

O símbolo `@` é apenas *syntactic sugar* para a reatribuição `alvo = decorador(alvo)`. A execução ocorre de baixo para cima ao empilhar múltiplos decoradores.

## 3. A Regra de Ouro

A função `wrapper` interna DEVE espelhar perfeitamente a interface (parâmetros, incluindo `self` em métodos, e o retorno) da função original.

## 4. Boas Práticas

Entenda a diferença crucial entre *Import Time* e *Runtime*. Sempre utilize `@wraps` do `functools` para não quebrar os metadados da função.